Praktikum 4: Ray Trace

In dieser vierten Praktikumsaufgabe ist im Vergleich zur letzten Aufgabe gerade in `vec3 Scene::traceRay(const Ray& ray, double IoR, int recDepth) const` sehr viel passiert . Leider hat der supertolle Entwickler es nicht geschafft das Raytracing vollständig umzusetzen da er beschäftigt damit war einen Adventskalender zu basteln. Vervollständigen Sie den Quelltext, damit der Raytracer jetzt zumindest sehr einfache Bilder rendern kann mitsamt Spiegelung und Krümmung durch transparente Objekte. Lesen Sie sich die neuen Kommentare im Quelltext durch oder nutzen Sie das Internet, um zu verstehen wie Spiegelung und Brechung sowie das Phong Beleuchtungsmodell (Phong Lighting) grob funktionieren sollen.

Viel Erfolg!

Bei Fragen oder Problemen schreiben Sie gerne im Moodle!

Aufgabe 1 — Vectoren

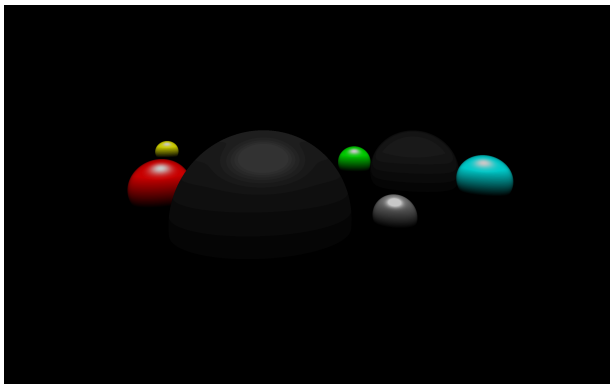


Figure 1: Das hier sollten sie sehen, wenn sie den Code ausführen ohne etwas zu ändern.

a) Öffnen Sie https://git.uni-due.de/vs.ude/objektorientierte-programmierung-cpp. Nutzen Sie git um das Repository zu *clonen* und importieren Sie das Projekt in 04_RayTrace/.

b) In `Vector3.cpp` finden Sie `vec3 vec3::reflection(const vec3 &normal) const`, welche benutzt wird um zu berechnen wie ein Vektor der anhand einer Oberflächennormale `normal` reflektiert wird. Das Prinzip ist recht einfach, wenn Sie einen Tennisball an eine Wand werfen, hängt es davon ab wie die Wand geneigt ist, um zu bestimmen wie der Ball nach dem Aufprall weiterfliegt. So funktioniert auch Tennis. Im Kommentar ist ein Hinweis dazu wie man mathematisch eine Reflektion berechnen kann. Wenn Sie es geschafft haben, können Sie in Scene.cpp `//return local_color * l + reflection * r;` anstatt des bisherigen Rückgabewertes zurückgeben und sollten dann ein Bild sehen das wie folgt aussieht:

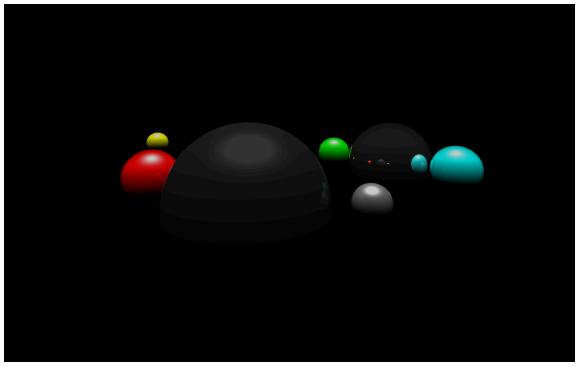


Figure 2: Das hier sollten sie sehen, wenn sie die Reflection richtig implementiert haben. Beachten Sie insbesondere die gespiegelten bunten Kugeln in der Spiegelkugel eher rechts im Bild. Spielen Sie gerne in der main.cpp mit den Werten der Kugeln herum um zu sehen wie sich Effekte ändern.

c) Glücklicherweise hat der Entwickler Ihnen `std::optional<vec3> vec3::refraction(const vec3 &normal, double IORRatio) const` bereits implementiert. Es ist nämlich etwas komplizierter als die Reflektion, da es sein kann dass durch **Total Internal Refraction** der Strahl in dem Material verschwindet. Dafür wurde aber nicht umgesetzt, dass die **refraction** Rays auch getraced werden. Mit `std::optional<vec3> refractionDir = ray._direction.refraction(normal, intersection->_material.getIndexOfRefraction());` erhalten Sie die Richtung, in die der Krümmungsstrahl zeigt. Glücklicherweise können Sie sich an `Ray reflectionRay(intersectionPoint + normal * epsilon, ray._direction.reflection(normal));` orientieren, wie ein `refractionRay` aussehen müsste. Vervollständigen Sie den Code und kommentieren Sie `//return local_color * l + refraction * t;` ein anstelle des bisherigen Rückgabewertes, um zu sehen ob Ihr Quelltext funktioniert. Wenn die Brechung richtig implementiert wurde, sieht es aus wie unten:

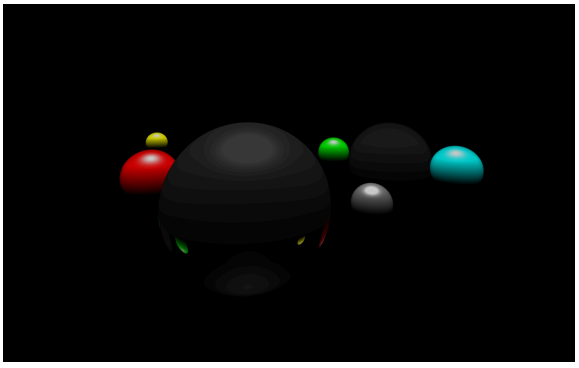


Figure 3: Das hier sollten sie sehen, wenn der refractionRay richtig implementiert wurde. Die Kugeln in der Glaskugel links befinden sich unten, das ist lustiger Lichtphysik geschuldet.

Wenn Sie reflection und refraction gelöst haben, können Sie `//return local_color * l + reflection * r + refraction * t;` einkommentieren und sehen ob ihr Bild genau wie unten aussieht:

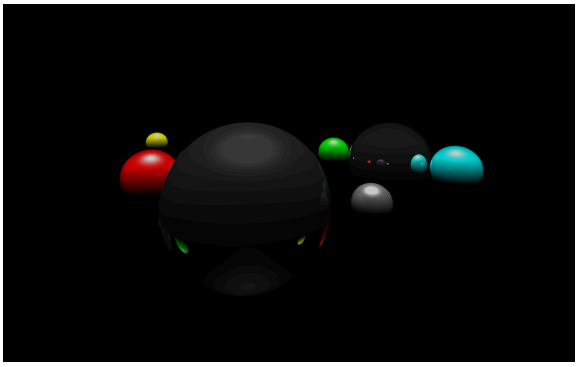


Figure 4: Das hier sollten sie sehen, wenn Sie fertig sind.

d) Hinweis zur Abgabe: Die Abgabe soll ein .zip Archiv sein, welches nur C++ Quelltext enthält und eine Datei aus der hervorgeht wer die Gruppenmitglieder sind mit Vorname, Nachname und Matrikelnummer.